1 Encounter

1.1 Principle Structure of Program

Definition 1.1.1 (Initialization): An initialization is a grammatical construct that augments a declaration and provides an initial value for the object.

```
size_t i = 5;
```

It's basically declaration + definition.

A more complex initialization is:

```
double A[5] = {
  [0] = 9.0,
  [1] = 2.9,
  [4] = 3.E+25,
  [3] = .00007,
};
```

This initializes the five items in A to the stated values. With order matching the index given.

□

Unlike object or function declaration, where several are allowed for the same identifier, definitions of objects or functions must be unique.

In other words, for a C program to be operational, any object or function used must have a definition (otherwise, the execution would not know where to look for them), and there must be no more than one definition (otherwise, the execution would become inconsistent).

□

Definition 1.1.2 (Statements): Statements are instructions that tell the compiler what to do with identifiers that have been declared so far.

E.g.,

```
for (size_t i = 0; i < 5; ++i) {
  printf("elements %zu is %g, \tits square is %g\n"),
  i,
  A[i],
  A[i]*A[i];
}

return EXIT_SUCCESS;
```

Some useful terms:

1. **for** statement
2. **domain iteration**
3. **loop body** (the code block in the {} in the above example, in general the code within {...} block that follows after **for** (...))
4. **loop variable**
5. **loop condition**

Remark: Domain iterations should be coded within a for statement

Remark: The loop variable should be defined in the initial part of a for

Definition 1.1.3 (Function Calls): Function calls are special statements that suspend the execution of the current function and then hand over control to the named function

In our example, the function is **printf**.

A function call also usually provides *arguments*.

These arguments are passed by value in C. I.e., the function can't mutate them.

Definition 1.1.4 (Call by Value): The mechanism where arguments are passed by value to the function in the function call. This means the function can't mutate the arguments.

We also have.

Definition 1.1.5 (Call by Reference): The mechanism where arguments are passed in such a way that the called function can change the value of the arguments

Function returns are the values a function returns. In C, the type of the function determines its return value. E.g.,

```
int main()
```

must return an integer.

```
bool is_long()
```

must return a boolean.

Once a function returns a value, it's execution stops and the process returns back to the original line of the function call.

□

Summary

1. C distinguishes the lexical structure (the punctuators, identifiers, and numbers), the grammatical structure (syntax), and the semantics (meaning) of programs
2. All identifiers (name) must be declared so we know the properties of the concept they represent
3. All objects (thing we deal with) and functions (methods we use to deal with things) must be defined; this is, we must specify how and where they came to be
4. Statements indicate how things are going to be done: iterations (**for**) repeat variations of certain tasks, function calls delegate a task to a function, and function returns go back to where came from.

2 Acquaintance

2.1 Everything is about control

C has five control statements:
1. **if** : introduces a *conditional execution* depending on a Boolean expression
2. **for**: iteration
3. **do**: iteration
4. **while**: iteration
5. **switch**: multiple selection based on an integer value

C has additional conditions: the ternary operator; the compile-time preprocessor conditionals #if, #ifdef, #ifndef, #elif, #elifdef, #elifndef, #lelse, #endif; and type generic expressions.

□

2.1.1 Conditional Execution

Definition 2.1.1.1 (**if**-statement): The statement of the form:

```
if (conditional) {
  // secondary block 0
}
else {
  // secondary block 1
}
```

The conditional (e.g., `i > 25`) is called the controlling expression, and the part in the `{ ... }` is called the secondary block.

The `if (...) ... else ...` is a selection statement. It selects one of the two possible code paths according to the contents of `( ... )`

> *Remark*: The value 0 represents logical false and any other value differing from 0 represents logical true.

The operators == and != represents “is equals to” and “not equals to” respectively.

Definition 2.1.1.2 (Boolean): Boolean values or bool is used to store truth values. It's values are true and false

Remark: In C, **bool**'s false is just 0 and its true is just the value of 1.

□

The operators == and != allow us to test for equality and inequality, respectively.

The code,

```
if (i != 0) {
  ...
}
```

as,

```
if (i) {
  ...
}
```

are equivalent.

Which one better is a question of coding style. The first one is easier for occasional reader of C whilst the later is often preferred in projects that assume knowledge about C.

Redundant comparisons quickly become unreadable and clutter the code. If you have a conditional that depends on a truth value, use the truth value directly as the condition. E.g

```
bool b = ...;
...
if ((b != false) == true) {
  ...
}
```

instead, use:

```
bool b = ...;
...
if (b) {
```

```
    ...
}
```

Remark: The above points can be summarized as saying: *Don't compare to 0, false, or true*

□

Since C considers all non-zero value to be true, we also have:

Remark: All scalars have a truth value

Here, we have a new term.

Definition 2.1.1.3 (Scalar): A scalar type include all numerical types such as size_t, bool, and int, and pointer.

□

2.1.2 Iterations

Definition 2.1.2.1 (The general form of for):

```
for (clause1; condition2; expression3) { secondary-block }
```

Here, `clause1` is usually an assignment expression or a variable definition. It serves to state an initial value for the iteration domain.

`condition2` tests whether the iteration should continue.

`expression3` updates the iteration variable used in `clause1`. It is performed at the end of each iteration.

Notes:

1. Because we want iteration variables to be defined narrowly in the context for a `for` loop, `clause1`, in most cases, be a variable definition.
2. Because `for` is relatively complex with its four different parts and not easy to capture visually, the secondary block should usually be a compound statement indicated by `{ ... }`.

Examples

```
for (size_t i = 10; i; --i) {
  something(i);
}

for (size_t i = 0, stop = upper_bound(); i < stop; ++i) {
  something_else(i);
}

for (size_t i = 9; i <= 9; --i) {
  something_else(i)
}
```

The first `for` counts `i` down from 10 to 1 inclusive. (Remember that 0 is always false for C, so `i` as condition is equivalent to `i == 0`).

In the second `for` loop, `clause1` declares and defines `i` and `stop`. The `condition2` loops till `i` reaches `stop`.

The third `for` looks like it would go on forever since if you're decreasing `i` from 9 then it will always satisfy `i <= 9`. However, the since `i` is a `size_t` it's never negative. Subtracting 1 from `i=0` would cause it to warp around to its maximum possible value. Thus, the loop will run for `i=0,...,9`.

□

There are two more iterative statements in C, `while` and `do`.

Definition 2.1.2.2 (While):

```
while (condition) secondary-block
```

Definition 2.1.2.3 (Do):

```
do secondary-block while (condition);
```

Examples

The following example shows a typical use of while. It implements Heron approximation to compute the multiplicative inverse of a number.

```
#include <tgmath.h>

constexpr double epsilon = 1E-9   // Desired precision
...
double const a = 34.0;
double x = 0.5;
while (fabs(1.0 - a*x) >= epsilon) { // Iterates until close
  x *= (2.0 - a*x);   // Heron approx
}
```

The program above iterates as long as the given condition evaluates true—as long as the absolute value of the difference between `1.0` and the computed product is smaller than `epsilon`, which represents the desired minimum precision of the computation.

The `do` loop is very similar, except that it checks the condition after the secondary block.

```
do { // Iterates
  x *= (2.0 - a*x); // Heron approximation
} while (fabs(1.0 - a*x) >= epsilon) // Iterates until close
```

So, if the condition immediately evaluates to `false`, a `while` loop will not run its secondary block at all, but the `do` loop will unconditionally run its block at least once.

□

All three iterations become more flexible with `break` and `continue` statements.

Definition 2.1.2.4 (`break`): A `break` statements stops the loop without re-evaluating the termination condition or executing the part of the secondary block after the `break` statement.

Examples

```
while (true) {
  double prod = a*x;
  if (fabs(1.0 - prod) < epsilon) { // Stops if close enough
    break;
  }
```

```
  x *= (2.0 - prod); // Heron Apprxoimation
}
for (;;) {
  double prod = a*x;
  if (fabs(1.0 - prod) < epsilon) { // Stops if close enough
    break;
  }
  x *= (2.0 - prod); // Heron Apprxoimation
}
```

Here, `for(;;)` is equivalent to `while (true)`. The fact that the controlling expression of `for` (the middle part between the `;;`) can be omitted and is interpreted as "always `true`" is just a historical artifact in the rules of C and has no other special purpose.

□

Definition 2.1.2.5 (`continue`): A `continue` skips the execution of the rest of the secondary block so all the statements in the compound statement after the continue are not executed for the current iteration. However, it then re-evaluates the condition and continues from the start of the secondary block if the condition is true.

Example

```
for (size_t i = 0; i < max_iteration; ++i) {
  if (x > 1.0) { // checks if we are on the correct side of 1
    x = 1.0/x;
  }
  double prod = a*x;
  if (fabs(1.0 - prod) < epsilon) { // stops if close enough
    break;
  }
  x *= (2.0 - prod); // Heron approximation
}
```

□

In the above examples, we used a standard macro `fabs`, which comes with the `<tgmath.h>` header.

It computes the absolute value of a double.

□

2.1.3 Multiple Selection

The last control statement that C has to offer is the switch statement, which is another selection statement. It is mainly used when cascades of `if-else` constructs would be too tedious.

Example

Instead of,

```
if (arg == 'm') {
  puts("this is a magpie");
} else if (arg == 'r') {
  puts("this is a raven");
} else if (arg == 'j') {
  puts("this is a jay");
} else if (arg == 'c') {
  puts("this is a chough");
} else {
  puts ("this is an unknown corvid");
}
```

we can use:

```
switch (arg) {
  case 'm': puts("this is a magpie");
            break;
  case 'r': puts("this is a raven");
            break;
  case 'j': puts("this is a jay");
            break;
  case 'c': puts("this is a cough");
            break;
  default: puts("this is an unknown corvid");
}
```

Here, we select one of the `puts` calls according to the value of the `arg` variable. Like `printf`.

Apparently, the moment a single argument is valid, the case breaks away and the program will just run through that without checking any further cases.

Thus, breaks are needed to make sure unintended behavior doesn't happen.

□

Definition 2.1.3.1 (`switch`): A switch statement is a control statement in C used for multiple comparisions. It's syntax is:

```
switch (expression) secondary-block
```

In the `secondary-block` it has cases which runs if it's true for the given `expression`. If no cases match, the `default` block runs.

Remark: Whilst the structure of a `switch` can be more flexible than `if-else`, it is restricted in that:

1. `case` values must be integer constant expressions
2. `case` values must be unique for each `switch` statement

2.2 Expression Computations

Definition 2.2.1 (Expression): Expressions are code snippets that compute a value based on other values.

The simplest expressions are arithmetic expressions.

□

2.2.1 Operands and Operators

Definition 2.2.1.1 (`size_t`): `size_t` is a type in C given by `<stdlib.h>` which corresponds to a non-negative integer. They correspond to "sizes."

Remark: `size_t` is finite and its upper limit is `SIZE_MAX`. Thus, `size_t` can represent value in the range:

```
[0, SIZE_MAX]
```

□

`SIZE_MAX` depends on the system. For modern systems,

$$2^{64} - 1$$

But for embedded systems, it can be as low as:

$$2^{16} - 1$$

□

The standard header `<stdint..h>` provides `SIZE_MAX` such that you don't have to figure out the valule yourself or specialize your program accordingly.

□

Definition 2.2.1.2 (Unsigned Integer Types): The concept of "numbers that cannot be negative" to which we referred for `size_t` corresponds to what C calls unsigned integer types.

Definition 2.2.1.3 (Operators and Operands): Symbols and combinations like + and `!=` are called operators, and the things which they are applied to care called opperands.

□

2.2.2 Arithmetic